Object Oriented Programming (CSEG1044)
Unit 06 – Lecture 01 Notes
Capstone Kickoff + Team Setup + Git/GitHub

Tofik Ali

February 27, 2026

Contents

1 Lecture Overview **2**

2 Syllabus Alignment (Unit 06) **2**

3 Learning Outcomes **2**

4 Project theme options (pick one) **2**

5 Scope boundaries (most important success factor) **3**

6 Evaluation mindset (what usually matters) **3**

7 Team setup checklist **3**

8 Git/GitHub workflow (simple and effective) **4**
- 8.1 Branching convention . . . 4
- 8.2 Pull request (PR) habit . . . 4
- 8.3 Commit message guideline . . . 4

9 How to write good GitHub issues (beginner-friendly) **4**
- 9.1 Issue template (simple) . . . 4
- 9.2 Example issue (Campus Resource Manager) . . . 5

10 Suggested repo structure (keep it simple) **5**

11 Deliverable-0 (proposal + backlog) **5**

12 In-Class Activity (evidence to produce) **6**

13 Checkpoint Questions (with sample answers) **6**

1 Lecture Overview

This is the start of the capstone project. The goal is not only "write code"; the goal is to build a small but complete software product with:

- clear scope,
- clean OOP design,
- a working Swing UI,
- JDBC persistence,
- and a professional Git/GitHub workflow.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

2 Syllabus Alignment (Unit 06)

Unit 06: standalone Java project; source control; collaboration; build and artifacts

3 Learning Outcomes

After this lecture, you should be able to:

- Select a realistic problem statement with clear scope boundaries.
- Create a team repo and organize work using issues and branches.
- Follow a simple Git workflow (branching + pull requests).
- Produce Deliverable-0: proposal (1 page) + initial backlog (issues).

4 Project theme options (pick one)

Choose a domain that naturally needs OOP modelling + collections + Swing UI + JDBC persistence:

- Library management (books, members, issue/return)
- Hostel / room booking
- Inventory management (products, stock in/out)
- Clinic appointment booking
- Campus resource booking (classrooms/labs/equipment) — recommended

5 Scope boundaries (most important success factor)

Projects fail mostly due to unclear scope. Use a simple scope definition:

- **Must-have**: required for minimum working demo.
- **Should-have**: improves product but not required for demo.
- **Could-have**: optional enhancements if time permits.
- **Non-goals**: explicitly out of scope (avoid wasted effort).

Example (Campus resource booking):

- Must: add/list resources; create booking; prevent overlapping bookings.
- Should: user login (simple roles); booking cancel; search/filter.
- Could: export report (CSV/PDF); notifications.
- Non-goals: full web app; payment gateway; complex authentication.

6 Evaluation mindset (what usually matters)

Even if the exact rubric differs, these are typical expectations:

- **Correctness**: features work, edge cases handled.
- **Design**: good class structure; separation of UI vs DB vs logic.
- **Database**: schema + CRUD, data persists correctly.
- **UI**: usable screens, validations, meaningful messages.
- **Git hygiene**: commits, branches, PRs, issues, evidence of collaboration.
- **Documentation**: README, setup steps, screenshots (optional), report/proposal.

7 Team setup checklist

1. Create team (3–4 students, as per batch rule).
2. Create a GitHub repository (one repo per team).
3. Add collaborators (all team members + instructor if needed).
4. Add a `README.md` with:
    - project title + short description,
    - tech stack (Java, Swing, JDBC, DB),
    - how to run (steps),

    - team members.

5. Create labels: `feature`, `bug`, `doc`, `db`, `ui`.

8 Git/GitHub workflow (simple and effective)

8.1 Branching convention

- `main`: always stable / demo-ready.
- Feature branches: `feature/<short-name>` (e.g., `feature/resource-crud`).
- Fix branches: `fix/<short-name>`.

8.2 Pull request (PR) habit

For each feature:

1. Create issue.
2. Create branch from `main`.
3. Commit frequently with meaningful messages.
4. Open a PR and request review from teammate.
5. Merge after review (and after fixing comments).

8.3 Commit message guideline

Keep it short and clear:

- `Add ResourceDao insert + findAll`
- `Fix booking overlap validation`
- `Update README setup steps`

9 How to write good GitHub issues (beginner-friendly)

GitHub issues are your "work tracker" and also your evidence of collaboration. An issue should be small enough to finish in **one focused session** (or at most 1–2 sessions).

9.1 Issue template (simple)

Use this structure (copy/paste into issue description):

- **Problem/Goal:** what needs to be built or fixed?
- **Acceptance criteria:** 3–6 testable bullets (how do we know it is done?)
- **Tasks:** checklist of steps (UI/service/DAO/DB/test/doc)

- **Notes:** screenshots, UI sketch, edge cases (optional)

9.2 Example issue (Campus Resource Manager)

Title: Prevent overlapping bookings for same resource

Acceptance criteria (sample):

- If a new booking overlaps an existing booking for the same resource, the system shows an error message and does not save it.
- Edge case: booking that ends at the exact start time of another booking is allowed.
- Valid booking is saved to DB and appears in booking list view.

Tasks (sample):

- Add overlap check method in `BookingService`
- Add DAO query to find overlaps (by resource + time range)
- Add UI validation message near booking form
- Test with 3 cases (overlap, boundary-touch, non-overlap)

10 Suggested repo structure (keep it simple)

You can organize your repo like this:

```
repo-root/
  README.md
  docs/
    deliverable-0-proposal.pdf
    deliverable-1-use-cases.pdf
    deliverable-2-class-diagram.pdf
  src/
    com/upes/crm/...
  db/
    schema.sql
```

Tip: Keep deliverables in `docs/` and keep code in `src/`. This makes demo preparation easier.

11 Deliverable-0 (proposal + backlog)

Deliverable-0A: 1-page proposal (template)

- Title + team members (roll numbers)
- Problem statement (3–5 lines)
- Must/Should/Could features (bullets)
- UI screens (list)

- Database entities/tables (list)
- Milestones (week-wise or lecture-wise plan)
- Risks (2–3) + mitigation
- GitHub repo link

Deliverable-0B: backlog (initial issues)

Create issues for each must-have item, plus technical tasks. Example issue list:

- Create DB schema (resources, bookings)
- Resource CRUD UI + DAO
- Booking create + validation (no overlap)
- List bookings (table view)
- Exception handling + user-friendly messages
- README run instructions

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

12 In-Class Activity (evidence to produce)

- Repo created + collaborators added
- README created
- At least 8–12 issues created (must-have + setup tasks)
- At least 1 PR opened (even if small: README + folder structure)

13 Checkpoint Questions (with sample answers)

Q1. Why do we define Must/Should/Could/Non-goals?

Sample answer: It prevents scope creep, helps teams finish a working minimum product, and makes planning measurable.

Q2. Why use branches + PRs instead of committing everything to `main`?

Sample answer: Branches isolate changes and keep `main` stable. PRs enable review and reduce bugs, and they create evidence of collaboration.